HOA SEN UNIVERSITY

www.hoasen.edu.vn

Laravel 3

CONTROLLER &
REQUESTS & RESPONSES

Content

- Controller
  - Defining Controllers
  - Single Action Controllers
  - Controller Middleware
  - Resource Controllers
    - Partial Resource Routes
    - Nested Resources
    - Naming Resource Routes
    - Naming Resource Route Parameters
  - Dependency Injection & Controllers
  - Route Caching
- HTTP Requests
- HTTP Responses

Defining Controllers

```
namespace App\Http\Controllers;
use App\Http\Controllers\Controller;
use App\User;
class UserController extends Controller {
 /**
* Show the profile for the given user.
 * @param int $id
* @return View
*/
 public function show($id) {
    return view('user.profile', ['user' =>
     User::findOrFail($id)]); }
}
```

```
Route::get('user/{id}', [UserController::class, 'show'] );
```

Controllers are not **required** to extend a base class. However, you will not have access to convenience features such as the `middleware`, `validate`, and `dispatch` methods.

Single Action Controllers

- If you would like to define a controller that only handles a single action, you may place a single __invoke method on the controller:

```
namespace App\Http\Controllers;
use App\Http\Controllers\Controller;
use App\User; class ShowProfile extends Controller {
/**
* Show the profile for the given user.
@param int $id
* @return View */
  public function __invoke($id) {
      return view('user.profile', ['user' =>
      User::findOrFail($id)]);
  }
}
```

```
Route::get('user/{id}', ShowProfile::class)
```

Controller Middleware

```
Route::get('profile',[UserController::class,'show']
->middleware('auth');
```

```
class UserController extends Controller
{
    /**
    * Instantiate a new controller instance.
    * @return void
    */
    public function __construct() {
        $this->middleware('auth');
        $this->middleware('log')->only('index');
        $this->middleware('subscribed')->except('store');
    }
}
```

```
    $this->middleware(function ($request, $next) {
     // ... return $next($request);
    });
```

Resource Controllers

- Laravel resource routing assigns the typical "CRUD" routes to a controller with a single line of code.

```
php artisan make:controller PhotoController --resource
```

- This command will generate a controller at app/Http/Controllers/PhotoController.php. The controller will contain a method for each of the available resource operations.

```
Route::resource('photos' , PhotoController::class);
```

```
Route::resources([ 'photos' => PhotoController::class,
                   'posts' => PostController::class]);
```

app/Http/Controllers/PhotoController.php

```php
namespace App\Http\Controllers;

use Illuminate\Http\Request;

class PhotoController extends Controller
{
    /** Display a listing of the resource. ...*/
    public function index(){...}

    /** Show the form for creating a new resource. ...*/
    public function create(){...}

    /** Store a newly created resource in storage. ...*/
    public function store(Request $request){...}

    /** Display the specified resource. ...*/
    public function show($id){...}

    /** Show the form for editing the specified resource. ...*/
    public function edit($id){...}

    /** Update the specified resource in storage. ...*/
    public function update(Request $request, $id){...}

    /** Remove the specified resource from storage. ...*/
    public function destroy($id){...}
}
```

Resource Controllers

- Actions Handled By Resource Controller

Verb	URI	Action	Route Name
GET	/photos	index	photos.index
GET	/photos/create	create	photos.create
POST	/photos	store	photos.store
GET	/photos/{photo}	show	photos.show
GET	/photos/{photo}/edit	edit	photos.edit
PUT/PATCH	/photos/{photo}	update	photos.update
DELETE	/photos/{photo}	destroy	photos.destroy

Specifying The Resource Model

```
php artisan make:controller PostController --resource --model=Post
```

```php
namespace App\Http\Controllers;

use ...

class PostController extends Controller
{
    /** Display a listing of the resource. ...*/
    public function index(){...}

    /** Show the form for creating a new resource. ...*/
    public function create(){...}

    /** Store a newly created resource in storage. ...*/
    public function store(Request $request){...}

    /** Display the specified resource. ...*/
    public function show(Post $post){...}

    /** Show the form for editing the specified resource. ...*/
    public function edit(Post $post){...}

    /** Update the specified resource in storage. ...*/
    public function update(Request $request, Post $post){...}

    /** Remove the specified resource from storage. ...*/
    public function destroy(Post $post){...}
}
```

```php
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Post extends Model
{
    //
}
```

Spoofing Form Methods

Spoofing Form Methods

Since HTML forms can't make `PUT`, `PATCH`, or `DELETE` requests, you will need to add a hidden `_method` field to spoof these HTTP verbs. The `@method` Blade directive can create this field for you:

```
<form action="/foo/bar" method="POST">
    @method('PUT')
</form>
```

Partial Resource Routes

```
Route::resource('photos', PhotoController::class)
    ->only(['index', 'show’]);
Route::resource('photos', PhotoController::class)
    ->except(['create', 'store', 'update', 'destroy’]);
```

API Resource Routes

- To quickly generate an API resource controller that does not include the create or edit methods, use the --api switch when executing the make:controller command:

```
php artisan make:controller API/PhotoController --api
```

```
Route::apiResources([
    'photos' => PhotoController::class,
    'posts' => PostController::class,
]);
```

Nested Resources

- Sometimes you may need to define routes to a nested resource. For example, a photo resource may have multiple comments that may be attached to the photo. To nest the resource controllers, use "dot" notation in your route declaration:

```
Route::resource('photos.comments',PhotoCommentController::class);
```

```
/photos/{photo}/comments/{comment}
```

- Often, it is not entirely necessary to have both the parent and the child IDs within a URI since the child ID is already a unique identifier. You may choose to use "shallow nesting":

```
Route::resource('photos.comments',
CommentController::class)->shallow();
```

Shallow Nesting Resources

```
Route::resource('photos.comments', CommentController::class);
```

GET|HEAD	photos/{photo}/comments	photos.comments.index
POST	photos/{photo}/comments	photos.comments.store
GET|HEAD	photos/{photo}/comments/create	photos.comments.create
DELETE	photos/{photo}/comments/{comment}	photos.comments.destroy
PUT|PATCH	photos/{photo}/comments/{comment}	photos.comments.update
GET|HEAD	photos/{photo}/comments/{comment}	photos.comments.show
GET|HEAD	photos/{photo}/comments/{comment}/edit	photos.comments.edit

```
Route::resource('photos.comments', CommentController::class)
->shallow();
```

POST	photos/{photo}/comments	photos.comments.store
GET|HEAD	photos/{photo}/comments	photos.comments.index
GET|HEAD	photos/{photo}/comments/create	photos.comments.create
PUT|PATCH	comments/{comment}	comments.update
GET|HEAD	comments/{comment}	comments.show
DELETE	comments/{comment}	comments.destroy
GET|HEAD	comments/{comment}/edit	comments.edit

Naming Resource Routes

- By default, all resource controller actions have a route name; however, you can override these names by passing a names array with your options:

```
Route::resource('photos', PhotoController::class)
        ->names([ 'create' => 'photos.build' ]);
```

- Naming Resource Route Parameters

```
Route::resource('users', AdminUserController::class)
          ->parameters([ 'users' => 'admin_user' ]);
```

URIs for the resource's show route

```
/users/{admin_user}
```

Dependency Injection & Controllers

```
namespace App\Http\Controllers;
use App\Repositories\UserRepository;
class UserController extends Controller {
    protected $users;
    public function __construct(UserRepository $users) {
        $this->users = $users;
    }
}
```

```
public function store(Request $request) {
    $name = $request->name;
    //
}
public function update(Request $request, $id) {
    //
}
```

Method Injection

If your controller method is also expecting input from a route parameter, list your route arguments after your other dependencies

Route Caching

Closure based routes cannot be cached. To use route caching, you must convert any Closure routes to controller classes.

```
php artisan route:cache
```

- Your cached routes file will be loaded on every request
- If you add any new routes you will need to generate a fresh route cache.
- Because of this, you should only run the route:cache command during your project's deployment.
- You may use the route:clear command to clear the route cache:

```
php artisan route:clear
```

HTTP REQUESTS

Accessing The Request

To obtain an instance of the current HTTP request via dependency injection, you should type-hint the Illuminate\Http\Request class on your controller method. The incoming request instance will automatically be injected by the service container:

- Dependency Injection & Route Parameters

```
public function store(Request $request) {
    $name = $request->input('name');
    //
}
```

Method of Controller

```
Route::put('user/{id}', [UserController::class, 'update']);
```

```
public function update(Request $request, $id)
{
}
```

- Accessing The Request Via Route Closures

Routes/web.php

```
use Illuminate\Http\Request;
Route::get('/', function (Request $request) {
 // accessing the request
});
```

Request Path & Method

- Retrieving The Request Path
  - The path method returns the request's path information. So, if the incoming request is targeted at http://domain.com/foo/bar, the path method will return foo/bar:

```
$uri = $request->path();
```

  - The is method allows you to verify that the incoming request path matches a given pattern

```
if ($request->is('admin/*')) {
    //
}
```

- Retrieving The Request URL

```
// Without Query String...
$url = $request->url();

// With Query String...
$url = $request->fullUrl();
```

```
$method = $request->method();

if ($request->isMethod('post')) {
    //
}
```

Retrieving Input (1)

- You may also retrieve all of the input data as an array using the all method

```
$input = $request->all();
```

- Regardless of the HTTP verb, the input method may be used to retrieve user input

```
$name = $request->input('name');
```

- You may pass a default value as the second argument to the input method

```
$name = $request->input('name', 'Sally');
```

- When working with forms that contain array inputs, use "dot" notation to access the arrays

```
<input name="products[0][name]">
```

```
$name = $request->input('products.0.name');
```

```
$names = $request->input('products.*.name');
```

- You may call the input method without any arguments in order to retrieve all of the input values as an associative array:

```
$input = $request->input();
```

Retrieving Input (2)

- Retrieving Input From The Query String

```
$name = $request->query('name');
$name = $request->query('name', 'Helen');
$query = $request->query();
```

- You may also access user input using dynamic properties on the Illuminate\Http\Request instance

```
$name = $request->name;
```

- Retrieving JSON Input Values
  - When sending JSON requests to your application, you may access the JSON data via the input method as long as the Content-Type header of the request is properly set to application/json. You may even use "dot" syntax to dig into JSON arrays:

```
$name = $request->input('user.name');
```

Retrieving Input (3)

- Retrieving Boolean Input Values

  When dealing with HTML elements like checkboxes, your application may receive "truthy" values that are actually strings.

  The boolean method returns true for 1, "1", true, "true", "on", and "yes". All other values will return false

```
$archived = $request->boolean('archived');
```

- Retrieving A Portion Of The Input Data

```
$input = $request->only(['username', 'password']);
$input = $request->only('username', 'password');
$input = $request->except(['credit_card']);
$input = $request->except('credit_card');
```

Retrieving Input (4)

- Determining If An Input Value Is Present

```
if ($request->has('name')) {
    //
}
```

```
$request->missing('name')
```

```
if ($request->has(['name', 'email'])) {...}
```

```
if ($request-> hasAny(['name', 'email'])) {...}
```

- If you would like to determine if a value is present on the request and is not empty, you may use the filled method:

```
if ($request->filled('name')) { ... }
```

Flashing Input To The Session (1)

- The flash method on the Illuminate\Http\Request class will flash the current input to the session so that it is available during the user's next request to the application:

```
$request->flash();
$request->flashOnly(['username', 'email']);
$request->flashExcept('password');
```

- Flashing Input Then Redirecting

```
return redirect('form')->withInput();

return redirect('form')->withInput(
    $request->except('password')
);
```

Flashing Input To The Session (2)

- Retrieving Old Input

  To retrieve flashed input from the previous request, use the old method on the Request instance. The old method will pull the previously flashed input data from the session:

```
$username = $request->old('username');
```

- Laravel also provides a global old helper. If you are displaying old input within a Blade template, it is more convenient to use the old helper. If no old input exists for the given field, null will be returned:

```
<input type="text" name="username"
                    value="{{ old('username') }}">
```

HTTP RESPONSES

Creating Responses

All routes and controllers should return a response to be sent back to the user's browser. Laravel provides several different ways to return responses.

- Strings & Arrays
  - The most basic response is returning a string from a route or controller. The framework will automatically convert the string into a full HTTP response

```
Route::get('/', function () {
    return 'Hello World';
});
```

  - The framework will automatically convert the array, eloquent collections into a JSON response

```
Route::get('/', function () {
    return [1, 2, 3];
});
```

Response Objects

- Returning a full Response instance allows you to customize the response's HTTP status code and headers. A Response instance inherits from the Symfony\Component\HttpFoundation\Response class, which provides a variety of methods for building HTTP responses.

```
Route::get('home', function () {
    return response('Hello World', 200)
        ->header('Content-Type', 'text/plain');
});
```

```
return response($content)
     ->header('Content-Type', $type)
     ->header('X-Header-One', 'Header Value’)
     ->header('X-Header-Two', 'Header Value');
```

```
return response($content)
    ->withHeaders([
        'Content-Type' => $type,
        'X-Header-One' => 'Header Value’,
        'X-Header-Two' => 'Header Value', ]);
```

Attaching Cookies To Responses

```
->cookie($name, $value, $minutes, $path, $domain, $secure,
$httpOnly)
```

```
return response($content)
        ->header('Content-Type', $type)
        ->cookie('name', 'value', $minutes);
```

- You can use the Cookie facade to "queue" cookies for attachment to the outgoing response from your application

```
Cookie::queue(Cookie::make('name', 'value', $minutes));
Cookie::queue('name', 'value', $minutes);
```

Other Response Types (1)

- View Responses

```
return response()
        ->view('hello', $data, 200)
        ->header('Content-Type', $type);
```

- JSON Responses
  - The json method will automatically set the Content-Type header to application/json, as well as convert the given array to JSON using the json_encode PHP function:

```
return response()
       ->json([ 'name' => 'Abigail', 'state' =>
       'CA', ]);
```

Other Response Types (2)

- File Downloads

```
return response()->download($pathToFile);

return response()->download($pathToFile, $name, $headers);

return response()->download($pathToFile)->deleteFileAfterSend();
```

- File Responses

  The file method may be used to display a file, such as an image or PDF, directly in the user's browser instead of initiating a download

```
return response()->file($pathToFile);

return response()->file($pathToFile, $headers);
```

Q&A